

SHA-256 Minimal Reflection Bundle — Teaching Notebook

This notebook is a **teaching companion** to the closure-law / crypto-algebra papers.

It is built to do four things clearly:

1. Reconstruct the **core SHA-256 algebra** from first principles.
2. Show the **sparse die / local reverse-closure** picture explicitly.
3. Build the **minimal reflection bundle** machinery:
 - mask nibble silhouette
 - (h)-nibble silhouette
 - (h)-chirality
4. Reproduce the **Depth-8 residual twin** and the **Depth-9 collapse** on real Bitcoin headers.

This notebook is intentionally self-contained and does **not** depend on external project files.

The formula visible in your screenshot

Yes — the formula in the screenshot is visible and is included here:

$$\Sigma_{t,j} = (c_{t,j}, q_{t,j}, s_t, m_t)$$

In the language of the paper:

- $(c_{t,j})$ is the local carry-state coordinate
- $(q_{t,j})$ is the local reflection / scar coordinate
- (s_t) is the Sziklai-side residue coordinate
- (m_t) is the schedule-law residue coordinate

In this notebook, we do not fully solve the general $(\Sigma_{t,j})$ -automaton.
But we do build the local pieces that make that state meaningful.

Notebook roadmap

- **Part A:** SHA-256 constants, bit operations, and single-block tracing
- **Part B:** exact reverse closure up to the fused wall
- **Part C:** staged carry masks, nibble silhouettes, and chirality maps
- **Part D:** real Bitcoin-header experiments
- **Part E:** Depth-8 residual twin and Depth-9 collapse

The notebook is designed so that each section can be run independently after the setup cells.

```
In [1]: # Optional install cell.  
# This notebook uses only the Python standard library.  
# If you want nicer tables, uncomment the next line.  
# %pip install pandas
```

```
In [2]: from __future__ import annotations  
  
from dataclasses import dataclass  
import hashlib  
import json  
import math  
import random  
import struct  
from functools import lru_cache  
from typing import Dict, List, Tuple
```

Part A — Core SHA-256 algebra

The SHA-256 round uses the standard ARX ingredients:

$$T1_t = h_t + \text{Sigma}_1(e_t) + \text{Ch}(e_t, f_t, g_t) + K_t + W_t \pmod{2^{32}}$$

$$T2_t = \text{Sigma}_0(a_t) + \text{Maj}(a_t, b_t, c_t) \pmod{2^{32}}$$

with

\$\$ $\Sigma_0(x) = \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x)$ \$\$

\$\$ $\Sigma_1(x) = \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x)$ \$\$

and message-schedule recurrence

\$\$ $W_t = \Sigma_1(W_{t-2}) + W_{t-7} + \Sigma_0(W_{t-15}) + W_{t-16} \pmod{2^{32}}$ \$\$

\$\$ $\Sigma_0(x) = \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x)$ \$\$

\$\$ $\Sigma_1(x) = \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x)$ \$\$

In [3]: MASK32 = 0xFFFFFFFF

```
H0 = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]
```

```
K = [
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90bffffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
]
```

```
def u32(x: int) -> int:
    return x & MASK32
```

```
def rotr(x: int, n: int) -> int:
    x &= MASK32
    return ((x >> n) | (x << (32 - n))) & MASK32
```

```
def ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & MASK32
```

```

def maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & MASK32

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def as_hex32(x: int) -> str:
    return f"0x{x & MASK32:08x}"

def hw(x: int) -> int:
    return (x & MASK32).bit_count()

```

```

In [4]: def words_from_block(block: bytes) -> List[int]:
        return list(struct.unpack(">16I", block))

def expand_schedule(w16: List[int]) -> List[int]:
    W = list(w16)
    for t in range(16, 64):
        W.append(u32(sigma1(W[t - 2]) + W[t - 7] + sigma0(W[t - 15]) + W[t - 16]))
    return W

def pad_sha256(msg: bytes) -> bytes:
    bit_len = len(msg) * 8
    out = msg + b"\x80"
    while len(out) % 64 != 56:
        out += b"\x00"
    out += struct.pack(">Q", bit_len)
    return out

def sha256_blocks(msg: bytes) -> List[bytes]:
    padded = pad_sha256(msg)
    return [padded[i:i+64] for i in range(0, len(padded), 64)]

```



```
def dbl_sha256_hex_display(msg: bytes) -> str:
    # Bitcoin display order = little-endian hex of the final digest
    return hashlib.sha256(hashlib.sha256(msg).digest()).digest()[::-1].hex()
```

A sparse die viewpoint

The round state is

$$x_t = \begin{bmatrix} a_t \\ b_t \\ c_t \\ d_t \\ e_t \\ f_t \\ g_t \\ h_t \end{bmatrix} \in (\mathbb{Z}/2^{32}\mathbb{Z})^8$$

and the sparse die equation is

$$x_{t+1} = P x_t + u_a(T1_t + T2_t) + u_e T1_t,$$

where only the (a)-lane and (e)-lane are non-linear reinjection seams.

Part B — Forward trace and exact reverse closure

The local reverse closure story is:

Given (x_{t+1}) , we recover by direct shift:

$$a_t = b_{t+1}, \quad b_t = c_{t+1}, \quad c_t = d_{t+1}$$

$$e_t = f_{t+1}, \quad f_t = g_{t+1}, \quad g_t = h_{t+1}$$

Then

$$T2_t = \Sigma_0(a_t) + \text{Maj}(a_t, b_t, c_t)$$

and from the forward equations

$$a_{t+1} = T1_t + T2_t, \quad e_{t+1} = d_t + T1_t$$

we get the Sziklai differential identity

$$a_{t+1} - e_{t+1} \equiv T2_t - d_t \pmod{2^{32}}$$

So

$$d_t \equiv T2_t - (a_{t+1} - e_{t+1}) \pmod{2^{32}}, \text{quad } T1_t \equiv e_{t+1} - d_t \pmod{2^{32}}$$

Everything closes locally except one fused ambiguity.

```
In [5]: def add32(a: int, b: int) -> Tuple[int, int]:
        total = (a & MASK32) + (b & MASK32)
        return total & MASK32, int(total >> 32)

        def carry_mask_add(a: int, b: int) -> int:
            a &= MASK32
            b &= MASK32
            carry = a & b
            union = carry
            s = a ^ b
            while carry:
                carry = (carry << 1) & MASK32
                newcarry = s & carry
                union |= newcarry
                s ^= carry
                carry = newcarry
            return union

        def nibble_hws(x: int) -> Tuple[int, ...]:
            return tuple(hw((x >> shift) & 0xF) for shift in range(0, 32, 4))
```

```
In [6]: @dataclass
        class RoundTraceFull:
            t: int
            a: int
            b: int
            c: int
            d: int
            e: int
            f: int
            g: int
            h: int
```

```

Wt: int
T1: int
T2: int
stage_carries: Tuple[int, int, int, int]
stage_masks: Tuple[int, int, int, int]
h_hw: int
h_nibble_hw: Tuple[int, ...]

```

```

In [7]: def compress_block_trace_full(block: bytes, state: List[int]) -> Dict[str, object]:
        W = expand_schedule(words_from_block(block))
        a, b, c, d, e, f, g, h = state
        traces: List[RoundTraceFull] = []

        for t in range(64):
            s1 = Sigma1(e)
            chv = ch(e, f, g)

            # Staged T1 additions
            sA, c1 = add32(h, s1)
            sB, c2 = add32(sA, chv)
            sC, c3 = add32(sB, K[t])
            T1, c4 = add32(sC, W[t])

            m1 = carry_mask_add(h, s1)
            m2 = carry_mask_add(sA, chv)
            m3 = carry_mask_add(sB, K[t])
            m4 = carry_mask_add(sC, W[t])

            T2 = u32(Sigma0(a) + maj(a, b, c))

            traces.append(
                RoundTraceFull(
                    t=t, a=a, b=b, c=c, d=d, e=e, f=f, g=g, h=h,
                    Wt=W[t], T1=T1, T2=T2,
                    stage_carries=(c1, c2, c3, c4),
                    stage_masks=(m1, m2, m3, m4),
                    h_hw=hw(h),
                    h_nibble_hw=nibble_hws(h),
                )
            )

        new_a = u32(T1 + T2)

```

```

        new_e = u32(d + T1)
        a, b, c, d, e, f, g, h = new_a, a, b, c, new_e, e, f, g

    state_out = [u32(state[i] + v) for i, v in enumerate([a, b, c, d, e, f, g, h])]
    return {
        "W": W,
        "traces": traces,
        "working_final": [a, b, c, d, e, f, g, h],
        "state_out": state_out,
    }

def sha256_trace_full(msg: bytes) -> List[Dict[str, object]]:
    blocks = sha256_blocks(msg)
    state = H0[:]
    out = []
    for idx, block in enumerate(blocks):
        step = compress_block_trace_full(block, state)
        out.append({
            "block_index": idx,
            "block": block,
            "init_state": state[:],
            **step,
        })
        state = step["state_out"]
    return out

```

In [8]:

```

def reverse_step_from_next(next_state: List[int], t: int, W_guess: int) -> Dict[str, object]:
    a1, b1, c1, d1, e1, f1, g1, h1 = next_state

    # Shift-lane recovery
    a_t = b1
    b_t = c1
    c_t = d1
    e_t = f1
    f_t = g1
    g_t = h1

    # Rebuild T2, then d_t, then T1
    T2 = u32(Sigma0(a_t) + maj(a_t, b_t, c_t))
    T1 = u32(a1 - T2)
    d_t = u32(e1 - T1)

```

```

# Fused-wall split using a supplied W guess
const_tail = u32(Sigma1(e_t) + ch(e_t, f_t, g_t) + K[t])
h_t = u32(T1 - const_tail - W_guess)

return {
  "state": [a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t],
  "T1": T1,
  "T2": T2,
}

```

The fused wall

After reverse-closing everything else, the unresolved local wall is

$$F_t := T1_t - \text{Sigma}_1(e_t) - \text{Ch}(e_t, f_t, g_t) - K_t \pmod{2^{32}}$$

so that

$$F_t \equiv h_t + W_t \pmod{2^{32}}.$$

This is the **fused wall**.

The point of the notebook is to show that we do **not** need to treat this as a blind 32-bit search. We can constrain it geometrically through local reflection classes.

Part C — Formal local observables

1. Nibble silhouette

For a 32-bit word (x) , define the (k) -th nibble extraction map by

$$\nu_k(x) = \text{HW}(\left\lfloor \left\lfloor x \gg 4k \right\rfloor \& 0xF \right\rfloor), \quad k=0, \dots, 7$$

and the full nibble silhouette

$\boldsymbol{\nu}(x) = \big(\nu_0(x), \dots, \nu_7(x)\big).$

2. Chirality map

Define the even/odd parity counts by

$\chi_{\mathrm{even}}(x) = \operatorname{HW}(x \ \&\ 0x55555555)$

$\chi_{\mathrm{odd}}(x) = \operatorname{HW}(x \ \&\ 0xAAAAAAAA)$

and

$\chi(x) = \big(\chi_{\mathrm{even}}(x), \chi_{\mathrm{odd}}(x)\big).$

3. Minimal reflection bundles

The two working bundles in this notebook are:

Local tie-break bundle: $B_t^{\star} = \big(\text{mask nibble silhouette}, \text{h nibble silhouette}, \text{h chirality}\big)$

and the shallower closure bundle: $B_t^{\{9\}} = \big(\text{mask nibble silhouette}, \text{h nibble silhouette}\big).$

```
In [9]: MASK_EVEN = 0x55555555
        MASK_ODD  = 0xAAAAAAAA

def nibble_silhouette(x: int) -> Tuple[int, ...]:
    return tuple(hw((x >> (4*k)) & 0xF) for k in range(8))

def chirality_map(x: int) -> Tuple[int, int]:
    return ((x & MASK_EVEN).bit_count(), (x & MASK_ODD).bit_count())

def stage_mask_nibble_bundle(stage_masks: Tuple[int, int, int, int]) -> Tuple[Tuple[int, ...], ...]:
    return tuple(nibble_silhouette(m) for m in stage_masks)
```

Part D — Real Bitcoin headers

We use two concrete examples:

- Bitcoin **Genesis** block header
- Bitcoin **Block 328734** header

The experiments in the paper revolve around the **tail rounds** and the question:

how small can the local reflection bundle get before it stops forcing a unique lawful chain?

```
In [10]: GENESIS_HEADER_HEX = (  
    "01000000"  
    + "00" * 32  
    + "3ba3edfd7a7b12b27ac72c3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a"  
    + "29ab5f49"  
    + "ffff001d"  
    + "1dac2b7c"  
    )  
  
    BLOCK_328734_HEADER_HEX = (  
        "02000000"  
        "b6ff0b1b1680a2862a30ca44d346d9e8"  
        "910d334beb48ca0c00000000000000000"  
        "9d10aa52ee949386ca9385695f04ede2"  
        "70dda20810decd12bc9b048aaab31471"  
        "24d95a54"  
        "30c31b18"  
        "fe9f0864"  
    )  
  
    REAL_HEADERS = {  
        "genesis": {  
            "height": 0,  
            "known_hash": "00000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",  
            "header": bytes.fromhex(GENESIS_HEADER_HEX),  
        },  
        "block_328734": {  
            "height": 328734,  
            "known_hash": "00000000000000009a11b3972c8e532fe964de937c9e0096b43814e67af3728",  
            "header": bytes.fromhex(BLOCK_328734_HEADER_HEX),  
        },  
    }
```

```
    },  
}
```

```
In [11]: for name, item in REAL_HEADERS.items():  
        calc = dbl_sha256_hex_display(item["header"])  
        print(name)  
        print(" matches known hash:", calc == item["known_hash"])  
        print(" hash:", calc)
```

genesis

matches known hash: True

hash: 00000000019d6689c085ae165831e934fff763ae46a2a6c172b3f1b60a8ce26f

block_328734

matches known hash: True

hash: 00000000000000009a11b3972c8e532fe964de937c9e0096b43814e67af3728

```
In [12]: header_data = {}  
        for name, item in REAL_HEADERS.items():  
            trace = sha256_trace_full(item["header"])  
            header_data[name] = {  
                "trace": trace,  
                "block1": trace[1], # the second block is where the 80-byte Bitcoin header finishes  
            }  
  
        for name in header_data:  
            block = header_data[name]["block1"]  
            print(name, "block index:", block["block_index"], "rounds:", len(block["traces"]))
```

genesis block index: 1 rounds: 64

block_328734 block index: 1 rounds: 64

Quick inspection of the tail round

We look at round (t=63) in the second block, because this is where the reverse tail experiments start.

```
In [13]: for name in header_data:  
        obs = header_data[name]["block1"]["traces"][63]  
        print(f"=== {name} / round 63 ===")  
        print("W63      =", as_hex32(obs.Wt))  
        print("T1       =", as_hex32(obs.T1))  
        print("T2       =", as_hex32(obs.T2))
```



```

print("h          =", as_hex32(obs.h))
print("h hw       =", obs.h_hw)
print("h nibbles   =", obs.h_nibble_hw)
print("h chirality =", chirality_map(obs.h))
print("stage carries =", obs.stage_carries)
print("stage mask nibs =", stage_mask_nibble_bundle(obs.stage_masks))
print()

```

=== genesis / round 63 ===

```

W63      = 0x86b0b8d5
T1        = 0x567f8aa5
T2        = 0x9c31de46
h         = 0xf6f1e66a
h hw      = 20
h nibbles = (2, 2, 2, 3, 1, 4, 2, 4)
h chirality = (9, 11)
stage carries = (1, 1, 0, 1)
stage mask nibs = ((1, 2, 1, 3, 1, 4, 0, 3), (0, 0, 4, 0, 1, 2, 4, 4), (3, 4, 1, 3, 1, 3, 0, 0), (0, 3, 1, 4, 0, 1, 4, 1))

```

=== block_328734 / round 63 ===

```

W63      = 0xa572aedd
T1        = 0x4213b0ed
T2        = 0x9e1cfb9a
h         = 0x2e591456
h hw      = 14
h nibbles = (2, 2, 1, 1, 2, 2, 3, 1)
h chirality = (9, 5)
stage carries = (0, 1, 1, 1)
stage mask nibs = ((0, 0, 0, 1, 2, 2, 2, 1), (0, 3, 2, 3, 0, 4, 3, 4), (3, 4, 2, 4, 4, 3, 2, 2), (0, 1, 3, 0, 0, 3, 3, 3))

```

Part E — Local enumeration under reflection bundles

The goal of the next cells is to treat the fused wall as a constrained carry-state object rather than a blind 32-bit unknown.

We enumerate local solutions that satisfy:

- stage-mask nibble silhouette

- optionally (h)-nibble silhouette
- optionally (h)-chirality

This is the local teaching version of the minimal reflection-bundle program.

```
In [16]: def enumerate_mask_nib_h(
    obs: RoundTraceFull,
    next_state: List[int],
    use_h_nib: bool = False,
    use_h_chir: bool = False,
    limit: int = 200000,
) -> List[Dict[str, object]]:
    a1, b1, c1_, d1, e1, f1, g1, h1 = next_state
    a_t = b1
    b_t = c1_
    c_t = d1
    e_t = f1
    f_t = g1
    g_t = h1

    T2 = u32(Sigma0(a_t) + maj(a_t, b_t, c_t))
    T1 = u32(a1 - T2)

    s1 = Sigma1(e_t)
    chv = ch(e_t, f_t, g_t)

    mask_target = stage_mask_nibble_bundle(obs.stage_masks)
    h_nib_target = nibble_silhouette(obs.h)
    h_chir_target = chirality_map(obs.h)

    sols: List[Dict[str, object]] = []

    def rec(j, c1, c2, c3, c4, nib_idx, n1, n2, n3, n4, hnib, he, ho, h, w):
        if len(sols) >= limit:
            return

        if j == 32:
            ok = (n1, n2, n3, n4) == (
                mask_target[0][7],
                mask_target[1][7],
                mask_target[2][7],
```

```

        mask_target[3][7],
    )
    if use_h_nib:
        ok &= (hnib == h_nib_target[7])
    if use_h_chir:
        ok &= ((he, ho) == h_chir_target)

    if ok:
        d_t = u32(e1 - T1)
        sols.append({
            "h": h,
            "W": w,
            "state": [a_t, b_t, c_t, d_t, e_t, f_t, g_t, h],
        })
    return

cur_nib = j // 4
if cur_nib != nib_idx:
    if (n1, n2, n3, n4) != (
        mask_target[0][nib_idx],
        mask_target[1][nib_idx],
        mask_target[2][nib_idx],
        mask_target[3][nib_idx],
    ):
        return
    if use_h_nib and hnib != h_nib_target[nib_idx]:
        return
    nib_idx = cur_nib
    n1 = n2 = n3 = n4 = 0
    hnib = 0

T1j = (T1 >> j) & 1
s1j = (s1 >> j) & 1
chj = (chv >> j) & 1
Kj = (K[obs.t] >> j) & 1
even = (j % 2 == 0)

for hj in (0, 1):
    # Stage 1
    s = hj + s1j + c1
    a = s & 1
    c1n = s >> 1

```

```

# Stage 2
s = a + chj + c2
b = s & 1
c2n = s >> 1

# Stage 3
s = b + Kj + c3
c = s & 1
c3n = s >> 1

# Stage 4 (fused wall split)
wj = T1j ^ c ^ c4
s = c + wj + c4
bit = s & 1
c4n = s >> 1
if bit != T1j:
    continue

nn1, nn2, nn3, nn4 = n1 + c1n, n2 + c2n, n3 + c3n, n4 + c4n
if (
    nn1 > mask_target[0][nib_idx]
    or nn2 > mask_target[1][nib_idx]
    or nn3 > mask_target[2][nib_idx]
    or nn4 > mask_target[3][nib_idx]
):
    continue

nh = hnib + hj
if use_h_nib and nh > h_nib_target[nib_idx]:
    continue

ne, no = he, ho
if even:
    ne += hj
else:
    no += hj
if use_h_chir and (ne > h_chir_target[0] or no > h_chir_target[1]):
    continue

rec(
    j + 1,

```

```

        c1n, c2n, c3n, c4n,
        nib_idx,
        nn1, nn2, nn3, nn4,
        nh,
        ne, no,
        h | (hj << j),
        w | (wj << j),
    )

    rec(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
    return sols

```

```

In [17]: def chain_tail(
    block: Dict[str, object],
    depth: int,
    use_h_nib: bool = False,
    use_h_chir: bool = False,
) -> List[Dict[str, object]]:
    t_lo = 64 - depth
    paths = [{"next_state": block["working_final"], "guesses": {}}]

    for t in range(63, t_lo - 1, -1):
        obs = block["traces"][t]
        new_paths = []
        for p in paths:
            sols = enumerate_mask_nib_h(
                obs,
                p["next_state"],
                use_h_nib=use_h_nib,
                use_h_chir=use_h_chir,
            )
            for sol in sols:
                d = dict(p["guesses"])
                d[t] = sol["W"]
                new_paths.append({
                    "next_state": sol["state"],
                    "guesses": d,
                })
        paths = new_paths
    return paths

```

Survivor counts across depths

The next cell measures the surviving chains under three bundles:

1. mask nibble silhouette only
2. mask nibble silhouette + (h)-nibble silhouette
3. mask nibble silhouette + (h)-nibble silhouette + (h)-chirality

```
In [18]: def survivor_table(name: str, depth_start: int = 8, depth_end: int = 12) -> List[Dict[str, object]]:
    block = header_data[name]["block1"]
    true = {t: block["W"][t] for t in range(64 - depth_end, 64)}
    rows = []
    for depth in range(depth_start, depth_end + 1):
        t_lo = 64 - depth
        true_sub = {t: true[t] for t in range(t_lo, 64)}
        for label, flags in [
            ("mask_nib", dict(use_h_nib=False, use_h_chir=False)),
            ("mask_nib+h_nib", dict(use_h_nib=True, use_h_chir=False)),
            ("mask_nib+h_nib+h_chir", dict(use_h_nib=True, use_h_chir=True)),
        ]:
            paths = chain_tail(block, depth, **flags)
            truth_hits = sum(1 for p in paths if p["guesses"] == true_sub)
            rows.append({
                "header": name,
                "depth": depth,
                "bundle": label,
                "survivors": len(paths),
                "truth_paths": truth_hits,
            })
    return rows

genesis_rows = survivor_table("genesis")
block_rows = survivor_table("block_328734")
genesis_rows + block_rows[:3]
```

```
Out[18]: [{'header': 'genesis',
            'depth': 8,
            'bundle': 'mask_nib',
            'survivors': 32,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 8,
            'bundle': 'mask_nib+h_nib',
            'survivors': 2,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 8,
            'bundle': 'mask_nib+h_nib+h_chir',
            'survivors': 1,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 9,
            'bundle': 'mask_nib',
            'survivors': 8,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 9,
            'bundle': 'mask_nib+h_nib',
            'survivors': 1,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 9,
            'bundle': 'mask_nib+h_nib+h_chir',
            'survivors': 1,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 10,
            'bundle': 'mask_nib',
            'survivors': 8,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 10,
            'bundle': 'mask_nib+h_nib',
            'survivors': 1,
            'truth_paths': 1},
          {'header': 'genesis',
            'depth': 10,
```

```
'bundle': 'mask_nib+h_nib+h_chir',
'survivors': 1,
'truth_paths': 1},
{'header': 'genesis',
'depth': 11,
'bundle': 'mask_nib',
'survivors': 32,
'truth_paths': 1},
{'header': 'genesis',
'depth': 11,
'bundle': 'mask_nib+h_nib',
'survivors': 1,
'truth_paths': 1},
{'header': 'genesis',
'depth': 11,
'bundle': 'mask_nib+h_nib+h_chir',
'survivors': 1,
'truth_paths': 1},
{'header': 'genesis',
'depth': 12,
'bundle': 'mask_nib',
'survivors': 128,
'truth_paths': 1},
{'header': 'genesis',
'depth': 12,
'bundle': 'mask_nib+h_nib',
'survivors': 1,
'truth_paths': 1},
{'header': 'genesis',
'depth': 12,
'bundle': 'mask_nib+h_nib+h_chir',
'survivors': 1,
'truth_paths': 1},
{'header': 'block_328734',
'depth': 8,
'bundle': 'mask_nib',
'survivors': 8,
'truth_paths': 1},
{'header': 'block_328734',
'depth': 8,
'bundle': 'mask_nib+h_nib',
'survivors': 1,
```



```
'truth_paths': 1},  
{ 'header': 'block_328734',  
  'depth': 8,  
  'bundle': 'mask_nib+h_nib+h_chir',  
  'survivors': 1,  
  'truth_paths': 1}]
```

```
In [19]: for title, rows in [("genesis", genesis_rows), ("block_328734", block_rows)]:  
         print(f"=== {title} ===")  
         for r in rows:  
             print(f"depth={r['depth']:>2} | {r['bundle']:<24} survivors={r['survivors']:<4} truth_paths={r['truth_paths']:  
         print()
```

```

=== genesis ===
depth= 8 | mask_nib                survivors=32  truth_paths=1
depth= 8 | mask_nib+h_nib          survivors=2   truth_paths=1
depth= 8 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth= 9 | mask_nib                survivors=8   truth_paths=1
depth= 9 | mask_nib+h_nib          survivors=1   truth_paths=1
depth= 9 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=10 | mask_nib                survivors=8   truth_paths=1
depth=10 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=10 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=11 | mask_nib                survivors=32  truth_paths=1
depth=11 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=11 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=12 | mask_nib                survivors=128  truth_paths=1
depth=12 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=12 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1

=== block_328734 ===
depth= 8 | mask_nib                survivors=8   truth_paths=1
depth= 8 | mask_nib+h_nib          survivors=1   truth_paths=1
depth= 8 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth= 9 | mask_nib                survivors=128  truth_paths=1
depth= 9 | mask_nib+h_nib          survivors=2   truth_paths=1
depth= 9 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=10 | mask_nib                survivors=64   truth_paths=1
depth=10 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=10 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=11 | mask_nib                survivors=64   truth_paths=1
depth=11 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=11 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1
depth=12 | mask_nib                survivors=16   truth_paths=1
depth=12 | mask_nib+h_nib          survivors=1   truth_paths=1
depth=12 | mask_nib+h_nib+h_chir   survivors=1   truth_paths=1

```

Expected teaching takeaway

The typical pattern is:

- the local bundle [$\big(\text{mask nibble silhouette}\big), \big(\text{h nibble silhouette}\big)$] is already very strong,

- and the full local tie-break bundle [$\big(\text{mask nibble silhouette}, \text{h nibble silhouette}, \text{h chirality}\big)$] collapses the tail even faster.

This is exactly the paper's "minimal reflection bundle" logic in executable form.

Part F — The Genesis residual twin

The key teaching moment is the **Depth-8 Genesis twin**.

Under

$\big(\text{mask nibble silhouette}, \text{h nibble silhouette}\big)$

the Genesis tail leaves **two** survivors at depth (8).

Those two survivors differ only at round (56).

We now isolate them.

```
In [20]: genesis_block = header_data["genesis"]["block1"]
g_paths_depth8 = sorted(
    chain_tail(genesis_block, depth=8, use_h_nib=True, use_h_chir=False),
    key=lambda p: p["guesses"][56],
)

len(g_paths_depth8), [as_hex32(p["guesses"][56]) for p in g_paths_depth8]
```

```
Out[20]: (2, ['0x6ec7e42f', '0x6ecee42f'])
```

```
In [21]: pair_analysis = []
for idx, p in enumerate(g_paths_depth8):
    w56 = p["guesses"][56]
    h56 = p["next_state"][-1]
    pair_analysis.append({
        "idx": idx,
        "w56_hex": as_hex32(w56),
        "h56_hex": as_hex32(h56),
        "h56_hw": hw(h56),
```

```

        "h56_nibbles": nibble_silhouette(h56),
        "h56_chirality": chirality_map(h56),
    })

```

```
pair_analysis
```

```

Out[21]: [{ 'idx': 0,
  'W56_hex': '0x6ec7e42f',
  'h56_hex': '0x4d8edce6',
  'h56_hw': 18,
  'h56_nibbles': (2, 3, 2, 3, 3, 1, 3, 1),
  'h56_chirality': (9, 9)},
  { 'idx': 1,
  'W56_hex': '0x6ecee42f',
  'h56_hex': '0x4d87dce6',
  'h56_hw': 18,
  'h56_nibbles': (2, 3, 2, 3, 3, 1, 3, 1),
  'h56_chirality': (10, 8)}]

```

```

In [22]: W0 = int(pair_analysis[0]["W56_hex"], 16)
W1 = int(pair_analysis[1]["W56_hex"], 16)
H0 = int(pair_analysis[0]["h56_hex"], 16)
H1 = int(pair_analysis[1]["h56_hex"], 16)

delta_xor = W0 ^ W1
same_sum = ((W0 + H0) & MASK32) == ((W1 + H1) & MASK32)

print("W56 xor delta:", as_hex32(delta_xor))
print("h56 xor delta:", as_hex32(H0 ^ H1))
print("same fused-wall sum:", same_sum)
print("true chirality :", pair_analysis[0]["h56_chirality"])
print("false chirality:", pair_analysis[1]["h56_chirality"])

```

```

W56 xor delta: 0x00090000
h56 xor delta: 0x00090000
same fused-wall sum: True
true chirality : (9, 9)
false chirality: (10, 8)

```

This is the structural heart of the residual twin.

The pair satisfies:

$$W_{56}^{(0)} \oplus W_{56}^{(1)} = 0x00090000$$

$$h_{56}^{(0)} \oplus h_{56}^{(1)} = 0x00090000$$

while preserving the fused-wall sum

$$(W_{56}^{(0)} + h_{56}^{(0)}) \bmod 2^{32} = (W_{56}^{(1)} + h_{56}^{(1)}) \bmod 2^{32}$$

So the false twin is not random.

It is a **balanced residual split** of the same fused wall.

What stays the same:

- local mass
- local nibble shape

What changes:

- **phase-handedness**

Part G — Depth-9 collapse

The final teaching point is that the false twin is **not globally stable**.

Under the same shallow bundle

$$\big(\text{mask nibble silhouette}\big) \text{h nibble silhouette}$$

we step one more round backward, from depth (8) to depth (9).

```
In [23]: obs55 = genesis_block["traces"][55]

continuations = []
for idx, p in enumerate(g_paths_depth8):
    sols55 = enumerate_mask_nib_h(
        obs55,
```

```

    p["next_state"],
    use_h_nib=True,
    use_h_chir=False,
)
continuations.append({
    "idx": idx,
    "W56_hex": as_hex32(p["guesses"][56]),
    "round55_extensions": len(sols55),
    "sample_W55": [as_hex32(s["W"]) for s in sols55[:8]],
})

```

continuations

```

Out[23]: [{ 'idx': 0,
  'W56_hex': '0x6ec7e42f',
  'round55_extensions': 1,
  'sample_W55': ['0x847d44e1'] },
{ 'idx': 1,
  'W56_hex': '0x6ecee42f',
  'round55_extensions': 0,
  'sample_W55': [] } ]

```

The teaching interpretation is direct:

- the true path still has a lawful round-55 continuation,
- the false twin becomes **non-extendable**.

That is why the Depth-9 collapse is stronger than a ranking result.

It is a **constructive shallow-depth closure** result.

Part H — Summary formulas and working closure law

Best current local bundle

The strongest current local teaching bundle is:

$$B_t^* = \big(\text{mask nibble silhouette}, \text{h nibble silhouette}, \text{h chirality} \big)$$

Shallower closure bundle

The weaker but still very powerful bundle is:

$$B_t^{(9)} = \big(\text{mask nibble silhouette}, \text{h nibble silhouette} \big)$$

which, for the tested headers in this notebook, collapses to a unique lawful chain by depth (9).

Residual functional viewpoint

Abstractly, one can think of the navigation problem as

$$B_t : \mathcal{P}_t \rightarrow \mathcal{G}_t$$

and

$$R_C(\mathbf{g}) = \sum_{t \in C} d(\left(B_t(\mathbf{g}_t), B_t^{\text{obs}}\right))$$

but in this notebook we went one step further and used **exact local acceptance under bundle constraints**, not just scoring.

Coupled state reminder

The paper's formal state reminder remains:

$$\Sigma_{t,j} = (c_{t,j}, q_{t,j}, s_t, m_t).$$

This notebook gave you executable local pieces of that object:

- $(c_{t,j})$ through staged carry logic,
- $(q_{t,j})$ through nibble/chirality reflection,
- and the beginnings of the coupled local closure law.

Part I — Suggested next experiments

1. Add a **schedule-residue** component explicitly and make (m_t) active.
2. Add an explicit **Sziklai zero-residue** tracker and make (s_t) active.
3. Test whether the same depth-9 collapse law holds on a wider sample of Bitcoin headers.
4. Try to derive weaker **admissible side proxies** that approximate the mask nibble silhouette without needing exact local mask access.
5. Build the next notebook around the full coupled lock-state automaton: $\Sigma_{t,j} = (c_{t,j}, q_{t,j}, s_t, m_t)$.

In []:

In []: